

one or more sort columns in their filter predicates ($\#UnionQueries$), and the average, minimum, maximum and median of the scan selectivities from scan fragments where at least one sort column appears as a filter, and the data types of the filter columns which are represented as a one-hot vector. The $\#dimensions$ in the one-hot vector is equal to $\#distinct$ data types of all the columns appearing in the table schema (note that a separate instance of the ML model is learned for each table). For multiple sort columns, we use AVG as the pooling technique to obtain the value of each feature dimension. In the case of bin-packing, there are no sort columns and hence, all the remaining dimensions in the sort vector are represented as 0. For the partitioning columns, we exclude the sorting technique dimension while retaining all the other dimensions shown in Figure 4 intact.

3.3.2 Creation of Target Labels for Training set. Section 3.3.1 details how the feature vectors are created but not the target labels. In this section, we discuss how we label the training set of feature vectors. Even a training set of layouts as small as $\sim 3\text{-}4\%$ of the entire candidate space incurs huge latencies to be evaluated on the original set of tables. Therefore, we run scan fragment queries on sampled tables to compute the skipping benefits of a data layout. Algorithm 3 details how the skipping benefits are estimated for an input data layout on its corresponding sample table. We downscale TFS and RGS by multiplying them with the convergence sampling rate from Algorithm 2 in Section 3.2.4 (which we use to create sampled tables) to obtain downsampledTFS and downsampledRGS (lines 1-3). If the partitioning column is non-null, we create a partitioned copy of the sample table (lines 4-6) using Create Table As Select (CTAS) on the original sample table. Next, we rewrite the sampled table using the sort scheme, downsampledTFS and downsampledRGS (line 7). Finally, we examine the EXPLAIN ANALYZE plan output from each of the scan fragment query workload to measure the skipping benefit as $(1.0 - \frac{scanFragment.rowsIn}{scanFragment.tableCardinality}) \times 100.0\%$ and it is used as the target label for the feature vector of the corresponding input data layout. Next, we train a gradient boosting tree (GBT) as a regression model on the labeled set of data layouts and use it as the prediction model to predict the skipping capability on the $\sim 96\text{-}97\%$ unlabeled set of data layouts. Finally, the best layout is applied on the original set of input tables to optimize them and yield the best possible skipping benefits on the input query workload. We also evaluated PTO on alternative ML baselines based on hyperparameter optimization (HPO) (Refer to Section 4.1.2).

4 Experiments

4.1 Experimental Setup

4.1.1 Datasets. We evaluate PTO on TPC-H [10, 15] and TPC-DS [16, 31] benchmarks and the corresponding query workloads at scale factor 10,000 (SF 10K)². We feed the 103 queries from TPC-DS (Q14, Q23, Q24 and Q39 are two-part queries among the 99 TPC-DS queries) and 22 queries from TPC-H as the input workload for which the layouts are optimized for the tables in these benchmarks.

4.1.2 Baselines. For the end-to-end evaluation of PTO (Section 4.2), we use the following baselines: **1. Default (Def_{opt})** : This is an implementation where the tables are not automatically optimized. The partitioning columns are borrowed from most common choices made by database engines for which configurations are posted on the public benchmark repositories [15, 16]. Among the TPC-DS tables, we partitioned *store_sales*, *catalog_sales* and *web_sales* on *ss_sold_date_sk*, *cs_sold_date_sk* and *ws_sold_date_sk* respectively. On TPC-H, we partitioned the *lineitem*, *orders*, *customer* and *part* tables on *l_shipdate*, *o_orderdate*, *c_mktsegment* and *p_brand* respectively. For TFS and RGS, we used the default sizes of the pre-created Parquet files from the dataset which we ingested into

²The table sizes shown in Table 2 may not add up to 10TB as expected for an SF 10K dataset due to compressed Parquet files in the Iceberg format. Same applies to the TPC-DS SF 30K table sizes in Table 6.

Iceberg tables. We did not specify any sort columns because that would require the suggestions from an automatic tool.

2. PTO using a human expert (PTO_{HumanExp}) : Since PTO co-optimizes the layout parameters using an ML model, we compare it with this baseline which instead uses a human expert. To this end, we re-use the top-k candidate generation mechanisms that we discussed for partitioning columns and sort schemes in Sections 3.2.2 and 3.2.3. We model the co-dependencies between parameters using a human expert who empirically comes up with heuristics detailed in Algorithm 4 while feeding each of the top-k candidate partitioning columns, *partitionCol*, as input.

Algorithm 4 *estimateFileSizes(table, partitionCol, TFS_{min}, TFS_{max}, RGS_{min}, RGS_{max}, files_{min}, files_{max}, partFiles_{min})*

```

1: parts  $\leftarrow$  partitionTable(table, partitionCol)
2: TFS  $\leftarrow \frac{\text{table.size}}{\min(\max(\text{parts}, \text{files}_{\max}), \max(\text{partFiles}_{\min} \times \text{parts}, \text{files}_{\min}))}$ 
3: TFS  $\leftarrow \min(\text{TFS}_{\max}, \max(\text{TFS}_{\min}, \text{TFS}))$ 
4: RGS  $\leftarrow \min(\text{RGS}_{\max}, \frac{\text{TFS}}{4}, \max(\text{RGS}_{\min}, 100\text{K} \times \frac{\text{table.size}}{|\text{table}|}))$ 
5: return (TFS, RGS)

```

- (1) This baseline, PTO_{HumanExp}, models the dependency between the partitioning column and TFS by ensuring that there are at least *partFiles_{min}* files per partition if the table is partitioned or at least *files_{min}* files if non-partitioned (line 2 in Algorithm 4).
- (2) The number of files is capped at *files_{max}* if the total number of partitions, *parts* $\leq$ *files_{max}* (line 2).
- (3) The target file size TFS is computed by dividing the size of the table in MB by the number of expected files (line 2).
- (4) We ensure that *TFS_{min}* $\leq$ *TFS* $\leq$ *TFS_{max}* (line 3).
- (5) We fit at least 100K rows in a row group or have 4 row groups in a file (line 4).
- (6) We empirically set *files_{min}*, *files_{max}* and *partFiles_{min}* to 30, 10K and 10 respectively. *TFS_{min}*, *TFS_{max}*, *RGS_{min}* and *RGS_{max}* are set to 32 MB, 512 MB, 8 MB and 128 MB respectively.
- (7) Finally, it scores the pre-identified *partitionCol*, *TFS* and *RGS* in conjunction with the top-k candidate sort schemes (binpacking is included when #candidates < k) on the sampled tables by computing the skipping benefits on each candidate layout to pick the best possible layout. It measures the skipping benefits using the downscaling method described in Section 3.3.2.

The baselines for the ML model, sort cluster size detection and candidate sort scheme discovery are discussed in Sections 4.3.1, 4.3.2 and 4.3.3. A weighted PTO baseline is evaluated in Section 4.3.4.

4.1.3 Configurations and Settings. We conducted our experiments on a cluster of four machines with an aggregate 0.7 TB RAM and a total of 320 CPUs (80 physical cores with a hyperthreading factor of 4) of 2 GHz Intel(R) Xeon(R) Gold 6138 running Red Hat Enterprise Linux 9.5. We used MinIO for S3 storage and implemented PTO using Python 3.7.3 and Java with openJDK 17.0.13. We used scikit-learn 0.24.2 and HyperOpt [8, 9] to implement K-medoids and HPO algorithms like genetic optimization and Bayesian optimization using Tree Parzen Estimators (TPE) and random search. The parameters for PTO are discussed below.

We set a conservative value of k=3 for the generation of top-k candidate partitioning columns and sort schemes. The purpose of candidate generation is to create a compact search space of data layouts for quick evaluation. Even with k=3, an exhaustive evaluation takes several days which makes larger values of k impractical. To generate candidate partitioning columns, we ensure that

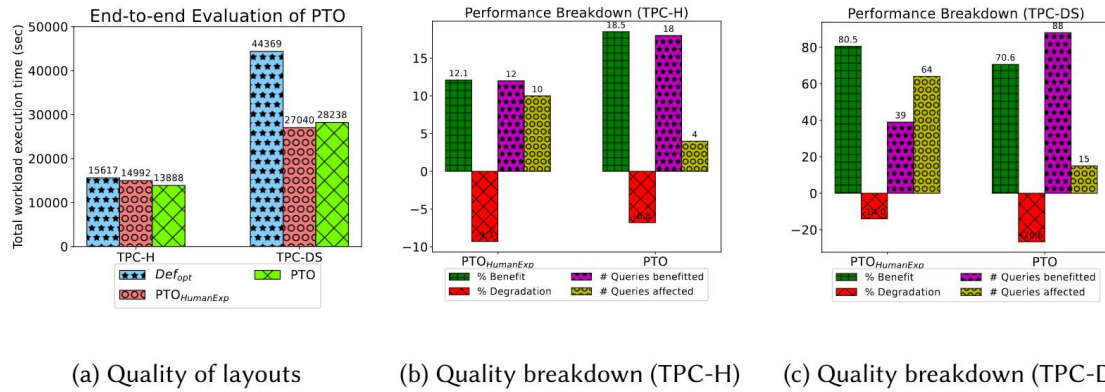


Fig. 5. PTO vs. $PTO_{HumanExp}$ and Def_{opt} on TPC-H/-DS workloads along with the performance breakdown.

the table has at least 10K rows (which is again a conservative estimate that leads to evaluating partitioning in almost all the cases) and only those columns which lead to at least 10 partitions and a maximum of 4,000 partitions are treated as candidate columns. Note that we also include “No Partitioning” as an additional candidate ($k=4$) to evaluate the effect of not partitioning the table. While generating the top- k candidate sort schemes using Algorithm 1, we set the query frequency threshold to 5% to ensure that any column appearing in very few queries is pruned away. We set TFS_{min} to 32 MB and TFS_{max} to 512 MB leading to $[32, 64, 128, 256, 512]$ as the discrete candidate set of TFS following the discussion from Section 3.2.5. For each candidate TFS, we have an RGS candidate set of $\frac{TFS}{2 \times i}$ with $i \in [1, 5]$ and an additional candidate of RGS = TFS i.e., a single row group per file. Top-4 partitioning columns, top-3 sort schemes, five candidates for TFS and six RGS candidates result in a test set of 360 candidate data layouts. The training set for the ML models is created from the top- k candidate partitioning columns and sort schemes along with random choices for TFS and RGS leading to 12 data layouts ($\sim 3.2\%$ of candidate space). We use them as fixed training and test sets for all our experiments.

4.2 End-to-end Evaluation

Here, we evaluate PTO end-to-end w.r.t. the entire query workload of TPC-H/DS at SF 10K.

4.2.1 Quality of the layouts discovered by PTO. In order to evaluate the quality of PTO, we run the query workload on the full set of tables in TPC-DS and TPC-H SF 10K benchmarks which are rewritten using the optimized layouts discovered by PTO. Note that we list the optimized layouts for all the 8 tables in TPC-H but in TPC-DS, we list the optimized layouts for the fact tables which are relatively large, i.e., sales, returns and inventory tables. Compared to the baseline of default layouts, Def_{opt} , PTO achieves 11% savings in latency over the end-to-end execution of the 22 TPC-H queries and 36% savings upon the 103 TPC-DS queries. This can be noticed in Figure 5a (13.8K seconds with PTO vs. 15.6K seconds using Def_{opt} on TPC-H and 28.2K seconds vs. 44.37 seconds on TPC-H). The reason for this improvement is evident from the optimized layouts shown in Table 2. All the tables are unsorted (use binpacking) in Def_{opt} and only a few large tables are partitioned such as the sales tables in TPC-DS and lineitem, customer, orders and part in TPC-H. In contrast, PTO uses candidate layouts in combination with gradient boosting trees (GBT) to co-discover optimal partitioning columns, TFS, RGS and the sort scheme. Z-ordering is predominantly used to sort on columns with large sort cluster sizes while benefitting maximum #queries.

Table 2. Optimized layouts configured by *Defopt*, *PTO_{HumanExp}* and *PTO* for the tables in TPC-H and TPC-DS.

Dataset	Table	Total Size	De _{opt}			PTO _{HumanExp}			PTO					
			Partitioning Column	TFS (MB)	RGS (MB)	Sort Scheme	Partitioning Column	TFS (MB)	RGS (MB)	Sort Scheme				
TPC-H (SF 10K)	customer lineitem	74.5 GB 1.6 TB	c_mktsegment l_shipdate	150 343	75 114	binpack binpack	c_nationkey l_commitdate	305 165	51 28	(c_mktsegment,c_acctbal) (l_shipdate,l_returnflag, l_shipmode,l_shipinstruct, l_discount)	c_nationkey l_shipdate	256 512	64 128	(c_mktsegment,c_acctbal) z-order(l_returnflag, l_shipmode,l_shipinstruct)
	nation orders part	2.4 KB 412.4 GB 41 GB	- o_orderdate p_brand	32 14 97	8 14 97	binpack binpack binpack	- o_orderdate p_size	32 42 84	8 11 21	n_name o_orderstatus z-order(p_brand,p_type, p_container)	- o_orderdate p_size	32 512 512	8 128 128	n_name o_orderstatus z-order(p_brand,p_type, p_container)
	partsupp region supplier	251 GB 1.3 KB 4.8 GB	- - -	29 32 43	29 8 43	binpack binpack binpack	- - s_nationkey	512 32 32	128 8 8	binpack binpack binpack	- - s_nationkey	512 32 512	128 8 128	binpack binpack binpack
	catalog_returns catalog_sales	83.8 GB 763 GB	- cs_sold_date_sk	97 128	97 128	binpack binpack	cr_returned_date_sk cs_sold_date_sk	32 86	8 22	binpack z-order(cs_bill_cdemo_sk, cs_bill_addr_sk, cs_sold_time_sk, cs_bill_customer_sk)	cr_returned_date_sk cs_sold_date_sk	128 512	32 128	(cr_item_sk,cr_return_amount) z-order(cs_bill_cdemo_sk, cs_bill_addr_sk, cs_sold_time_sk, cs_bill_customer_sk)
	inventory store_returns	5.9 GB 111 GB	- -	128 150	128 75	binpack binpack	inv_date_sk sr_returned_date_sk	32 32	4 8	binpack (sr_reason_sk, sr_return_amt,sr_cdemo_sk)	inv_date_sk sr_returned_date_sk	512 256	128 64	inv_quantity_on_hand sr_reason_sk
TPC-DS (SF 10K)	store_sales	893.8 GB	ss_sold_date_sk	323	108	binpack	ss_sold_date_sk	123	31	ss_promo_sk z-order(ss_store_sk, ss_addr_sk,ss_cdemo_sk, ss_promo_sk)	ss_sold_date_sk	512	128	ss_store_sk
	web_returns web_sales	39.2 GB 535.5 GB	- ws_sold_date_sk	28 43	28 43	binpack binpack	wr_returned_date_sk ws_sold_date_sk	32 37	8 9	wr_refunded_addr_sk z-order(ws_sold_time_sk, ws_bill_addr_sk, ws_ship_addr_sk, ws_ship_hdemo_sk)	wr_returned_date_sk ws_sold_date_sk	512 512	128 128	wr_refunded_addr_sk z-order(ws_sold_time_sk, ws_bill_addr_sk, ws_ship_addr_sk, ws_ship_hdemo_sk)

Human expert heuristics can be treated as gold-standard ground truth for a workload. They were discovered upon running the workload in an ad hoc, randomized way extensively with various parameter configurations. Although the human expert heuristics themselves are not parameterized by information from the query workload, the values of $files_{min}$, $files_{max}$, $partFiles_{min}$ and 100K rows per row group in Algorithm 4 of our paper are specific to SF 10K TPC-H/DS. Since $PTO_{HumanExp}$ forgoes the usage of GBT and instead replaces ML models with manually/empirically discovered heuristics to capture the co-dependencies among layout parameters, it is expected to discover an optimal layout. We can notice how PTO generically pushes TFS and RGS to be as large as possible while $PTO_{HumanExp}$ finds tailored TFS and precise RGS for each table. However, the drawback of $PTO_{HumanExp}$ is that the discovery of these heuristics takes several weeks and a plethora of experimental runs without a principled approach. Additionally, when the dataset and workload changes, re-discovering the optimal layout needs a new set of ad hoc runs. Figures 5b and 5c show that PTO benefits more queries (18 vs. 12 on TPC-H and 88 vs. 39 on TPC-DS) than $PTO_{HumanExp}$ despite being fully automatic and achieves almost comparable quality in terms of the latency benefits on the end-to-end query workload. This is because PTO shortlists candidate sort columns based on their scan selectivity and #queries they appear in as discussed in Algorithm 1.

Table 3. Latency breakdown for offline execution of PTO.

(a) Component-wise latencies.

Component	TPC-H (#Queries:22)	TPC-DS (#Queries:103)
Scan fragment extraction	1.43 sec	21.9 seconds
Sort cluster size estimation	1.89 hours	3.29 hours
Top-k candidate layout generation	0.006 seconds	33 seconds
Feature vector creation	3,763 seconds	1,581 seconds
Target label (data skipping) estimation	6.93 hours	29.6 hours
Applying the optimized layout	3.26 hours	2.74 hours

(b) Table-wise breakdown of latency-intensive components.

Dataset	Table	# Scan Fragment Queries	Sort cluster size estimation	Data skipping est (sample)			Apply optimized layout
				# Labeling iterations	Rewrite time/iter	Scoring time/iter	
TPC-DS	store_sales	66	1,416 sec	12	341 sec	4,753 sec	3,600 sec
	store_returns	15	1,233 sec		176 sec	117 sec	420 sec
	catalog_sales	37	884 sec		344 sec	1,360 sec	3,300 sec
	catalog_returns	10	273 sec		143 sec	41 sec	246 sec
	web_sales	35	7,007 sec		353 sec	539 sec	2,160 sec
	web_returns	10	963 sec		201 sec	38 sec	120 sec
	inventory	6	89 sec		116 sec	17 sec	28 sec
TPC-H	lineitem	5	4,088 sec	12	197 sec	214 sec	7,401 sec
	customer	4	1,235 sec		103 sec	16 sec	1,860 sec
	orders	3	221 sec		325 sec	132 sec	1,380 sec
	part	7	1,230 sec		110 sec	49 sec	480 sec
	partsupp	0	-		-	-	480 sec
	supplier	5	15 sec		92 sec	10 sec	113 sec
	nation	4	4 sec		-	-	17 sec
	region	3	3 sec		-	-	13 sec

4.2.2 Latency of PTO to generate the optimized layout. Table 3 shows the latencies incurred by PTO to discover the optimized layout. As we can notice from the component-wise latencies in Table 3a, the most expensive step is the generation of target labels for training feature vectors using the estimation of data skipping benefits detailed in Algorithm 3. Despite using sampling, it is latency inducing because each of the 12 training layouts needs to be evaluated over the scan fragment queries. Table 3b shows the time taken per labeling iteration and we see that evaluating one layout

takes 4,753 seconds for the TPC-DS store_sales table alone. One way to optimize this latency is to reduce the number of scan fragment queries used to estimate skipping benefits. We empirically show how we can achieve this without sacrificing layout quality in Section 4.5. Sort cluster size estimation is another such latency-intensive step for which we use an optimized approach of iterative sampling with the Good-Turing estimator. Since layout discovery is an offline step, it can be run in the background asynchronously.

4.3 Evaluating the components in PTO

In this section, we evaluate the components of PTO such as the predictive model, sort cluster size estimation, candidate sort scheme discovery. We also evaluate the impact of prioritizing long-running queries using query weighting on the layout quality.

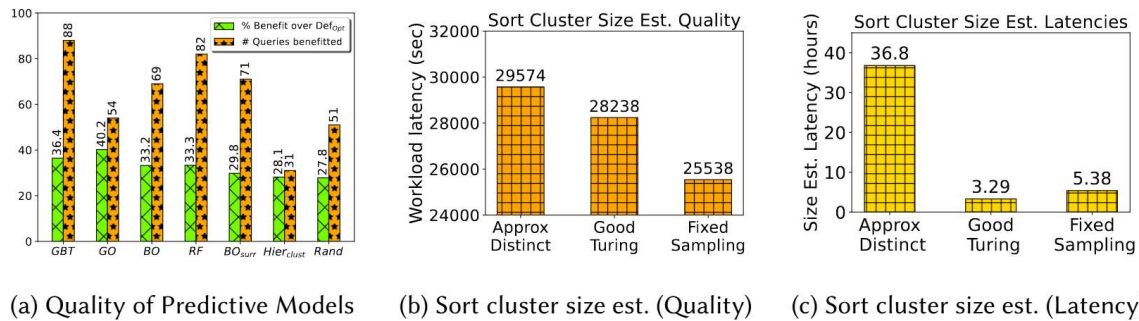


Fig. 6. Evaluating the quality of predictive models and sort cluster size estimation (TPC-DS SF 10K).

4.3.1 Evaluation of the Predictive Model. Here, we evaluate ML-based parameter co-detection models in PTO. Figure 6a evaluates the quality of the predictive models used in PTO and Table 4 shows the time taken to label the training samples. Since GBT and random forest (RF) are supervised models, they are trained in a single iteration using the 12 labeled examples (~3-4% of the candidate search space). Note that the actual training time of the model is insignificant but as listed in Table 3, label generation is the most expensive step. In fact, hyperparameter optimization (HPO) models such as genetic optimization (GO), Bayesian optimization (BO), hierarchical clustering (Hier_{clust}), and random selection using Bayesian models need 20 more training labels (an extra ~5-6% of the candidate space). All the HPO baselines including random search score one data layout per iteration thus labeling 20 extra training samples in 20 iterations. This is because they use 12 pre-trained layouts as bootstrapped data and need more iterations to detect the actual layout. That leads to a whopping overall labeling latency of 75.89 hours for the 32 labeling iterations in total for such alternative models which is why we use GBT as the default predictive model.

Figures 6a shows that the only HPO model benefiting from this additional training is GO which shows a 40.2% savings in latency over Def_{opt} at the expense of an additional 46 hours of labeling time. Interestingly, GO benefits 54 queries while 88 queries are benefitted by GBT. Hybrid baseline, BO_{surr}, using surrogate RF model runs for 380 iterations but scores only 20 training samples. In the remaining 360 iterations, it uses surrogate scores from RF. BO_{surr} performs worse than BO showing the futility of surrogate labels.

4.3.2 Evaluating Sort Cluster Size Estimation. Here, we show how replacing the default sort cluster size estimator, i.e., Good-Turing using iterative sampling (Algorithm 2) with alternative estimators based on APPROX DISTINCT and fixed sampling affects the quality and estimation latency.

Table 4. Labeling latencies for model training (TPC-DS).

Model Name	GBT	RF	GO/BO	Hier _{clust}	Rand	BO _{surr}
# Labeling Iterations	12			32		
# Training Iterations	1			20		380
Overall Latency		29.6 hours		75.89 hours		

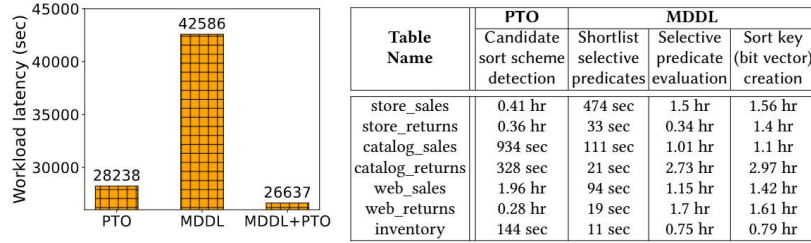


Fig. 7. Evaluating data layouts from PTO vs. MDDL [19]

- (1) **APPROX DISTINCT**: We run APPROX DISTINCT [21] on the full table to compute # sort clusters and the average distinct sort cluster size is estimated as $\frac{tableCardinality}{\#SortClusters}$.
- (2) **FIXED SAMPLING**: We take the upper bound of the sampling rate (0.1%) used in our Good-Turing implementation and repurpose it as fixed sampling rate on each table to measure the number of sort clusters and average distinct sort cluster size.

As mentioned in Section 3.2.4, Good-Turing can be replaced with any decent alternative estimator in PTO. APPROX DISTINCT is directly applied on the full set of tables without sampling and is expected to be a more accurate estimator than Good-Turing and fixed sampling-based estimators. However, Figure 6b shows that APPROX DISTINCT fails to fetch better query latency savings than Good-Turing despite spending $6.8\times$ more time estimating the cluster sizes. On the contrary, fixed sampling estimator at 0.1% sample rate yields the best latency savings. Upon inspecting the detected layouts, we found that the #queries benefited from each discovered sort strategy seem to impact the layout quality. This emphasizes the importance of co-optimizing both sort cluster size and the expected #queries benefited from each layout. Note that fixed sampling takes $1.6\times$ longer than Good-Turing as shown in Figure 6c to estimate the sort cluster sizes because iterative sampling using Good-Turing converges much sooner than 0.1% sample rate used by fixed sampling.

4.3.3 Comparison with MDDL [19]. In this section, we compare PTO against multidimensional data layout (MDDL) [19] from Amazon Redshift to evaluate candidate sort scheme discovery (discussed in Section 3.2.3). Ding et al. [19] characterize a data layout only based on the sort scheme and do not discover other parameters unlike PTO. Instead of using columns to sort the table, the top-k most selective filter predicates from the query workload are represented as a multidimensional bit vector which serves as the sort key. If a tuple satisfies a predicate, the corresponding bit is set. As we can see in Figure 7, the table layouts detected by MDDL result in $1.5\times$ longer workload execution latencies (42.5K seconds vs. 28.2K seconds) compared to PTO. Additionally, MDDL takes up to $62.6\times$ longer than PTO to detect the sort scheme. To give MDDL added advantage, we implement a variant MDDL+PTO which uses the optimal partitioning column, TFS and RGS discovered by PTO in combination with the sort strategy discovered by MDDL to get 6% better latency savings (26.6K seconds vs. 28.2K seconds) than PTO. In the original implementation of MDDL [19], atomic predicates are used to represent each bit in the sort key which we found to be more time-consuming to evaluate over large tables and can lead to scalability issues. Hence, we give MDDL an additional

advantage of directly using the filter predicates that we extract from the query workload as sort dimensions unlike Z-ordering which uses the filter columns as sort dimensions.

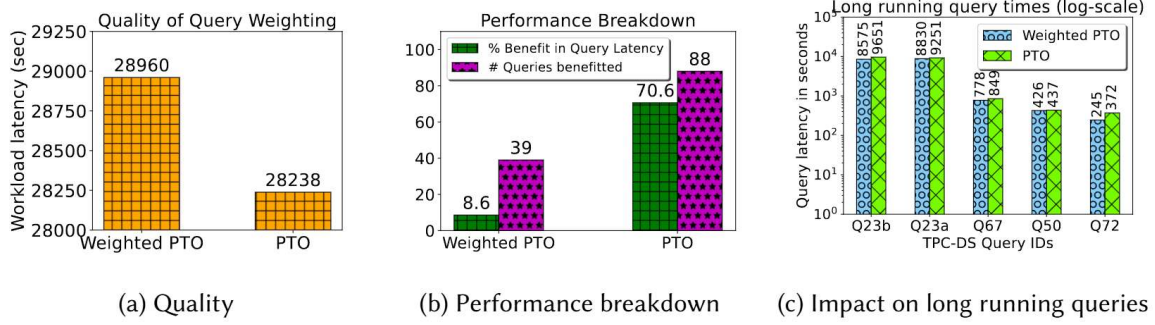


Fig. 8. Evaluation of query weighting (TPC-DS SF 10K).

4.3.4 Impact of Query Weighting. Here, we evaluate the impact of weighting queries from the workload during candidate sort scheme selection on the overall quality of the discovered data layouts. In real query workloads, end users may implicitly prioritize some queries over others by running them more frequently and it make sense to bias the layouts to benefit such queries. In the case of TPC-DS, there is no notion of how frequently each query is run. Therefore, we use the normalized latency of each query i.e., $\frac{queryLatency - minLatency}{maxLatency}$ (where min and max latencies are computed across all queries) $\in [0, 1]$ as its weight. Figures 8a, 8b show that Weighted PTO performs 2.5% worse than PTO w.r.t. overall savings in workload latency and benefits only 39 queries by 8.6% over Def_{opt} . This is because query weighting is inherently biased towards high latency queries. Interestingly, the top-5 long running queries benefitted by Weighted PTO (Figure 8c) consist of TPC-DS Q23a and Q23b which run for over 2.5 hours. Upon these two queries, Weighted PTO yields 4% and 11% latency savings compared to PTO.

4.4 Ablation Study and Sensitivity Evaluation

In this section, we report the results of an ablation study evaluating the isolated benefit of optimizing each data layout parameter one at a time. The second evaluation studies the sensitivity of PTO to hyperparameters such as top-k and candidate TFS, RGS values.

4.4.1 Isolated Benefit of each Layout Parameter. Figure 9a shows % improvement over Def_{opt} as well as the #queries benefitted by optimizing each parameter in isolation for TPC-DS. As we can notice, optimizing the partition column (*PartCol*) alone shows the least improvement of 3.9% and over fewest queries (52 out of 103 TPC-DS queries). This is because the largest of the fact tables (store_sales, web_sales, catalog_sales) are already partitioned by Def_{opt} . PTO additionally partitions store_returns, web_returns, catalog_returns and inventory which benefits fewer queries such as Q83 involving filtered scans on one or more of these tables. As expected, optimizing the sort schemes (*SortCol*) results in the most dominant latency reduction among all parameters by 26% while speeding up 88 queries. However, sorting alone falls short of the maximum possible benefit (36%) PTO can achieve by optimizing ALL parameters as shown in the figure. This establishes the importance of co-optimizing data layout parameters in PTO. Likewise, optimizing TFS alone gains only 5% because larger file sizes (512 MB) with smaller row groups (28 MB, 43 MB) created several (12 to 18) row groups for tables like web_sales and web_returns which was counterproductive. Since row groups were parameterized based on file sizes (by having 1 to 10 row groups per file), we

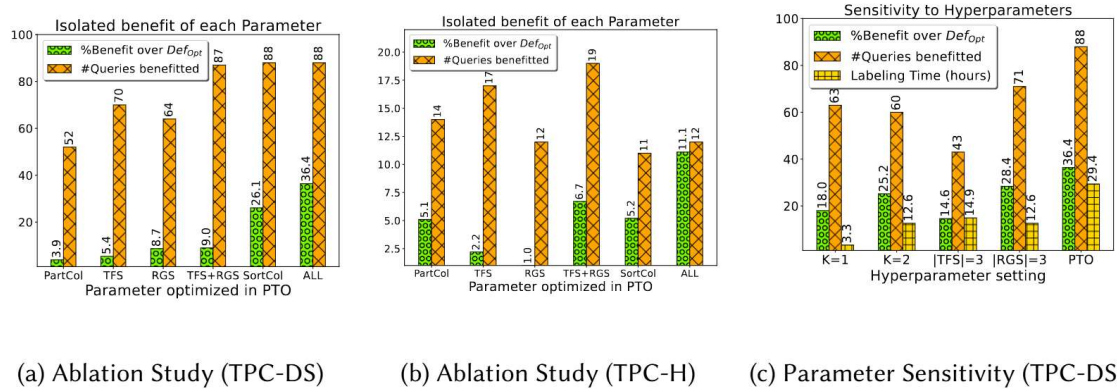


Fig. 9. Isolated Benefit of Each Parameter (Ablation Study) and Sensitivity to Hyperparameters

evaluated optimizing both TFS and RGS. Interestingly, TFS+RGS benefitted more queries but showed comparable improvement ($\sim 9\%$) to optimizing RGS alone. This explains that tuning TFS:RGS ratio is more important than tuning TFS alone as the overhead spent in reading metadata for several row groups negates the compaction gains from large TFS. Tuning RGS sets optimal #row groups for store_sales and catalog_returns thus benefitting 64 queries.

Figure 9b also shows that optimizing ALL parameters is crucial to get the highest savings of 11% over Def_{Opt} upon the TPC-H benchmark. Interestingly, optimizing TFS or RGS alone did not yield enough savings but co-optimizing TFS and RGS achieves the highest standalone benefit of 6.7%. This explains why increasing TFS alone with smaller row groups is not beneficial consistent with our observation on TPC-DS. Increasing RGS alone benefitted TPC-DS but did not make a difference for most TPC-H tables as it did not change #row groups per file. For instance, increasing row group size from 29 MB to 128 MB does not make a difference on a file sized 29 MB because the file has a single row group in both the settings. TFS+RGS also surpassed individually optimizing sort columns or partitioning column thus establishing the need to co-discover parameters. This is because Def_{Opt} uses lower TFS, RGS values and compacting them with suitable larger sizes fetches more benefit than isolated sorting or partitioning which is logically equivalent to a single-column sort. Isolated optimization sometimes benefitted more queries (14 to 19) compared to co-optimization benefitting 12 out of 22 TPC-H queries but the overall latency reduction is better with co-optimization. We treat the findings on TPC-DS more representative than TPC-H due to its comprehensive workload.

4.4.2 Sensitivity to Hyperparameters. PTO is most sensitive to the size of the shortlisted search space which is determined by top-k partitioning columns, top-k sort candidates and the discrete set of target file sizes (TFS). Therefore, in this section, we will study the impact of the size of the search space on PTO. More specifically, the goal of this experiment is to evaluate the quality of the data layouts PTO can detect from hyperparameter settings that create a smaller search space. Enlarging the shortlisted search space will further increase the optimization time of PTO. Instead of enlarging search space (more particularly for TFS), we can alter it for larger scale factors (scalability experiment in Section 4.5). Varying ML model hyperparameters did not have a significant impact due to an already compact search space. Instead of top-k=3 which is the default setting in PTO, we used top-k={1, 2} for candidate partitioning columns and sort strategies. We also included a comparison while reducing #TFS choices from 5 to 3 candidates (32 MB, 128 MB, 512 MB) and #RGS to 3 candidates resulting in 1, 4 and 10 row groups per file to evaluate the impact of shortlisting a smaller candidate space (fewer than 3 candidates is too tiny to get meaningful insights).

Table 5. Search space sizes for Hyperparameter Settings.

Setting	Top-k=1	Top-k=2	TFS=3	RGS=3	PTO
# Candidates Per Table	60	180	216	180	360

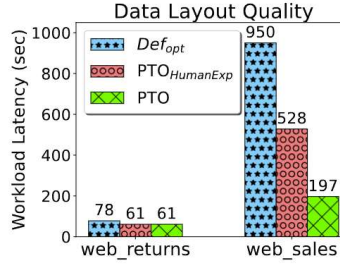


Fig. 10. Scalability Evaluation (TPC-DS SF 30K)

Table 5 shows the candidate space size per table for each evaluated hyperparameter setting, including the default search space sized 360 for PTO. Setting top-k=1 produces a candidate space sized (top-1 sort strategy $\times$ 5 TFS $\times$ 6 RGS per TFS $\times$ (top-1 partitioning + No Partitioning as an additional candidate)) thus yielding 60 candidates in the search space. Figure 9c and Table 5 show that top-k=2 yields a 25% reduction in workload execution latency over Def_{opt} with a shortlisted search space of 180 candidates which is better than 18% gain at top-k=1. Setting $|RGS|=3$ fetches a higher benefit of 28% despite equally shortlisting 180 candidates. This is because $|RGS|=3$ still preserves the three most important RGS choices without pruning any of the top-k sort schemes or partitioning columns. Thus, PTO is highly sensitive to top-k and also to TFS because $|TFS|=3$ reduced the latency gain to 14% despite shortlisting 216 candidates. Upon closer examination, we found that the lack of diverse TFS candidates in the training data did not give enough signal to PTO about optimal file sizes and row groups to select because RGS values are also pivoted on TFS. Since we choose $\sim 3\%$ of the candidate space for training, lower hyperparameters led to better labeling (skipping benefit estimation) times than vanilla PTO.

4.5 Scalability Evaluation w.r.t. Table Sizes

In this section, we show that PTO can scale to larger tables. To enable this evaluation on our cluster of 4 machines, we chose TPC-DS web_sales and web_returns at SF 30K as they are a representative pair of <sales, returns> fact tables. Since we cannot run the end-to-end TPC-DS queries on a controlled schema with two fact tables, we used the scan fragment queries on corresponding tables as the query workload to compare PTO with Def_{opt} . To accommodate growing table sizes and keep search space compact, we altered candidate TFS space from [32, 64, 128, 256, 512] MB to [64, 128, 256, 512, 1024] MB. We applied SF 10K PTO_{HumanExp} heuristics at 30K and set $files_{min}$ and $files_{max}$ to 64 MB and 1024 MB in Algorithm 4. Table 6 shows that PTO found larger TFS, RGS for web_sales but smaller TFS, RGS for web_returns at SF 30K than 10K due to the larger 0.1% sample drawn at SF 30K. This is because PTO estimates skipping benefits of layouts on sample tables which cannot be computed effectively unless the downscaled TFS and RGS can pack sufficient #rows in each rewritten row group or Parquet file in the sample. This phenomenon led to several downscaled candidate TFS and RGS configurations not resulting in enough row groups or target files to produce meaningful skipping benefits. Since samples are tiny at SF 10K for the relatively smaller web_returns table, we found that TFS and RGS were artificially pushed to larger values to